

🔥 CASTO_DE_GALLOSAPP API - Documentación Completa

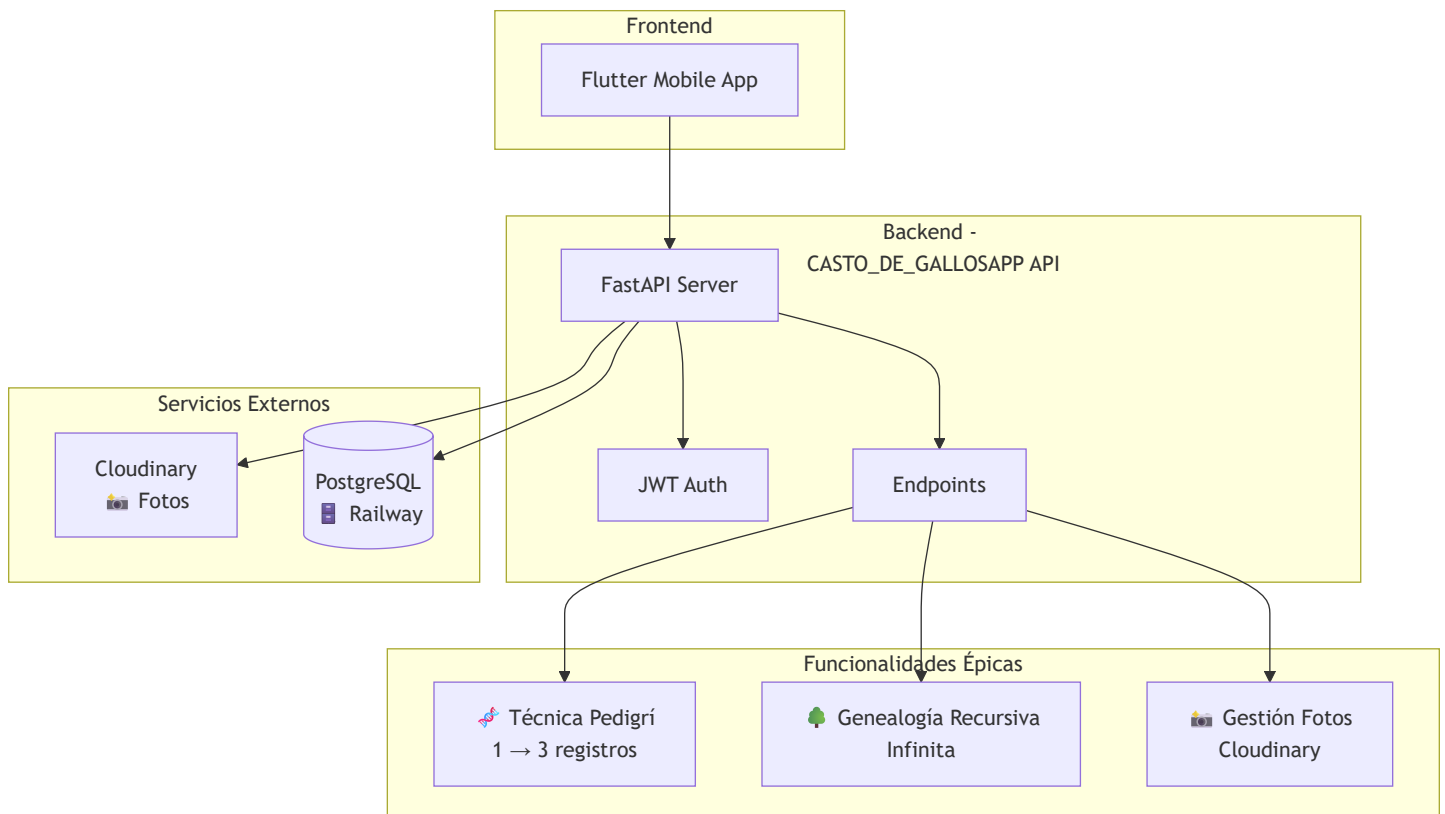
Sistema profesional para gestión de gallos de pelea

Desarrollado por el equipo de Casto de Gallos 🐓

📌 Ubicación del Proyecto

📁 C:\Users\acairamp\Documents\proyecto\Curso\Flutter\galloapp_backend\

🏗️ Arquitectura del Sistema



Estructura de Archivos

Modelos (SQLAlchemy)

```
└─ app/models/
  │ └─ __init__.py          # ✅ Imports corregidos
  │ └─ user.py              # Usuario base
  │ └─ profile.py           # Perfil de usuario
  │ └─ raza_simple.py       # Razas de gallos
  │ └─ gallo_simple.py      # 🔥 MODELO ÉPICO - Técnica Recursiva
└─ app/models_init.py      # 🛠️ Fix SQLAlchemy imports
```

Endpoints (FastAPI)

```
└─ app/api/v1/
  │ └─ auth.py              # Autenticación JWT
  │ └─ profiles.py          # Perfiles + Avatar Cloudinary
  │ └─ gallos_real.py       # CRUD básico gallos
  │ └─ gallos_con_pedigri.py # 🧬 TÉCNICA PEDIGRÍ ÉPICA
  │ └─ fotos_final.py       # 📷 Fotos Cloudinary
  │ └─ gallos_genealogia_epica.py # 🔥 Endpoint épico (corregido)
  │ └─ fotos_gallos_epica.py # 📷 Fotos épicas avanzadas
```

Servicios (Business Logic)

```
└─ app/services/
  │ └─ auth_service.py      # Lógica autenticación
  │ └─ profile_service.py   # Lógica perfiles
  │ └─ cloudinary_service.py # 📷 Gestión Cloudinary
  │ └─ genealogy_service.py # 🧬 Técnica recursiva genealógica
  │ └─ validation_service.py # ✅ Validaciones robustas
```

Schemas (Pydantic)

```
└─ app/schemas/
  │ └─ auth.py              # Schemas autenticación
  │ └─ profile.py           # Schemas perfiles
  │ └─ gallo.py             # Schemas gallos y genealogía
```

⚙ Configuración

```
|— app/core/
|   |— config.py           # Settings JWT/DB/Cloudinary
|   |— security.py        # JWT + Hash passwords
|— app/database.py        # Conexión PostgreSQL Railway
|— app/main.py            # 🚀 FastAPI app principal
|— .env                   # Variables de entorno
|— requirements.txt       # Dependencias Python
|— railway.toml           # Configuración Railway
|— Procfile               # Railway deploy config
```

🔒 Diagrama de Flujo de Autenticación

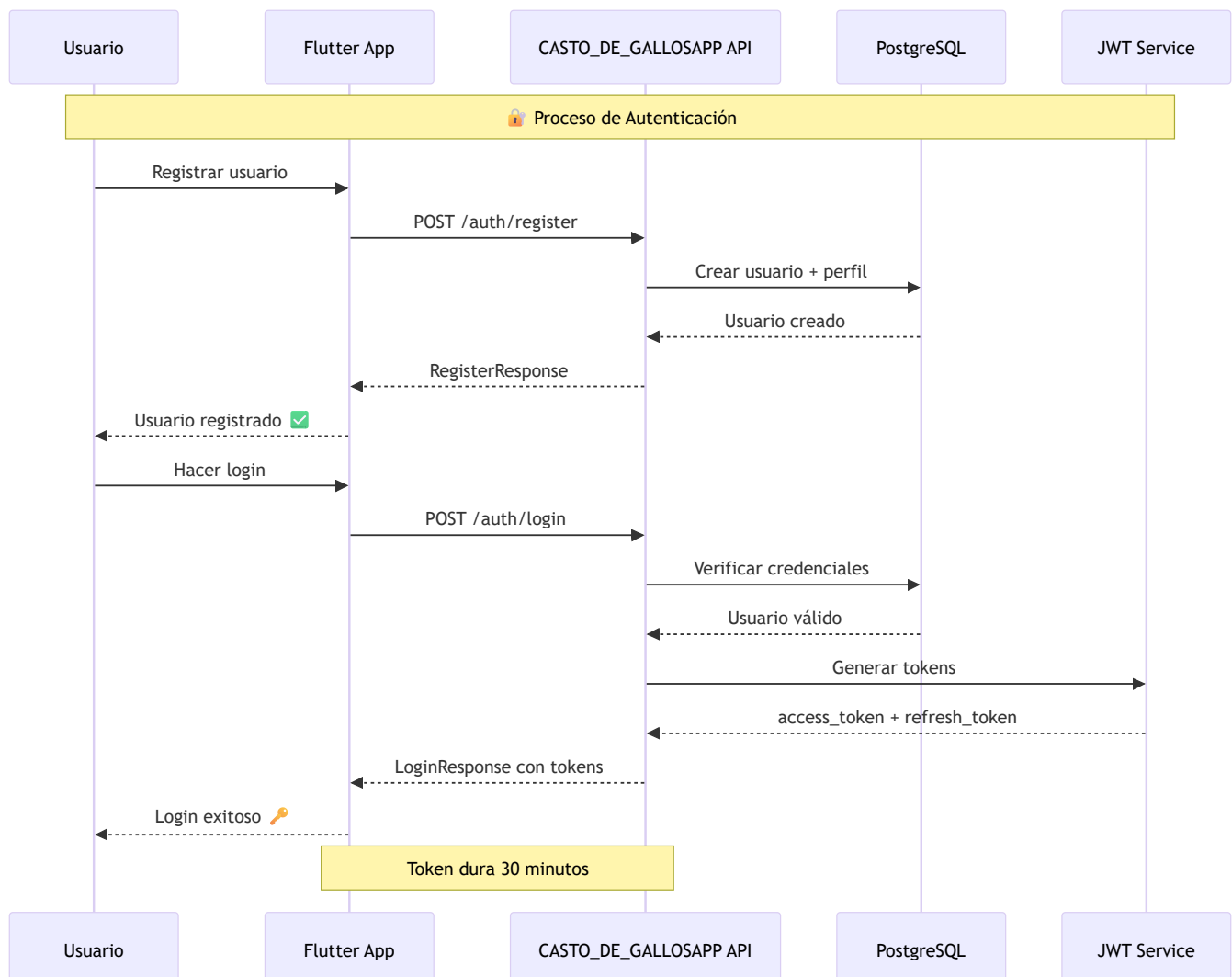




Diagrama de Técnica Pedigrí Épica

 Usuario crea gallo con pedigrí

 Datos del gallo
+ crear_padre=true
+ crear_madre=true

 Validar datos
códigos únicos

 Generar
id_gallo_genealogico

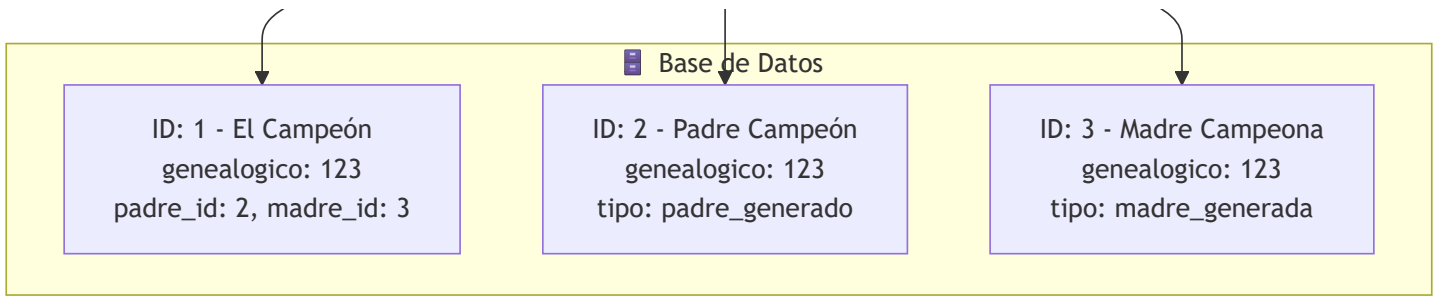
 Crear gallo principal
tipo: 'principal'

 Crear padre
tipo: 'padre_generado'

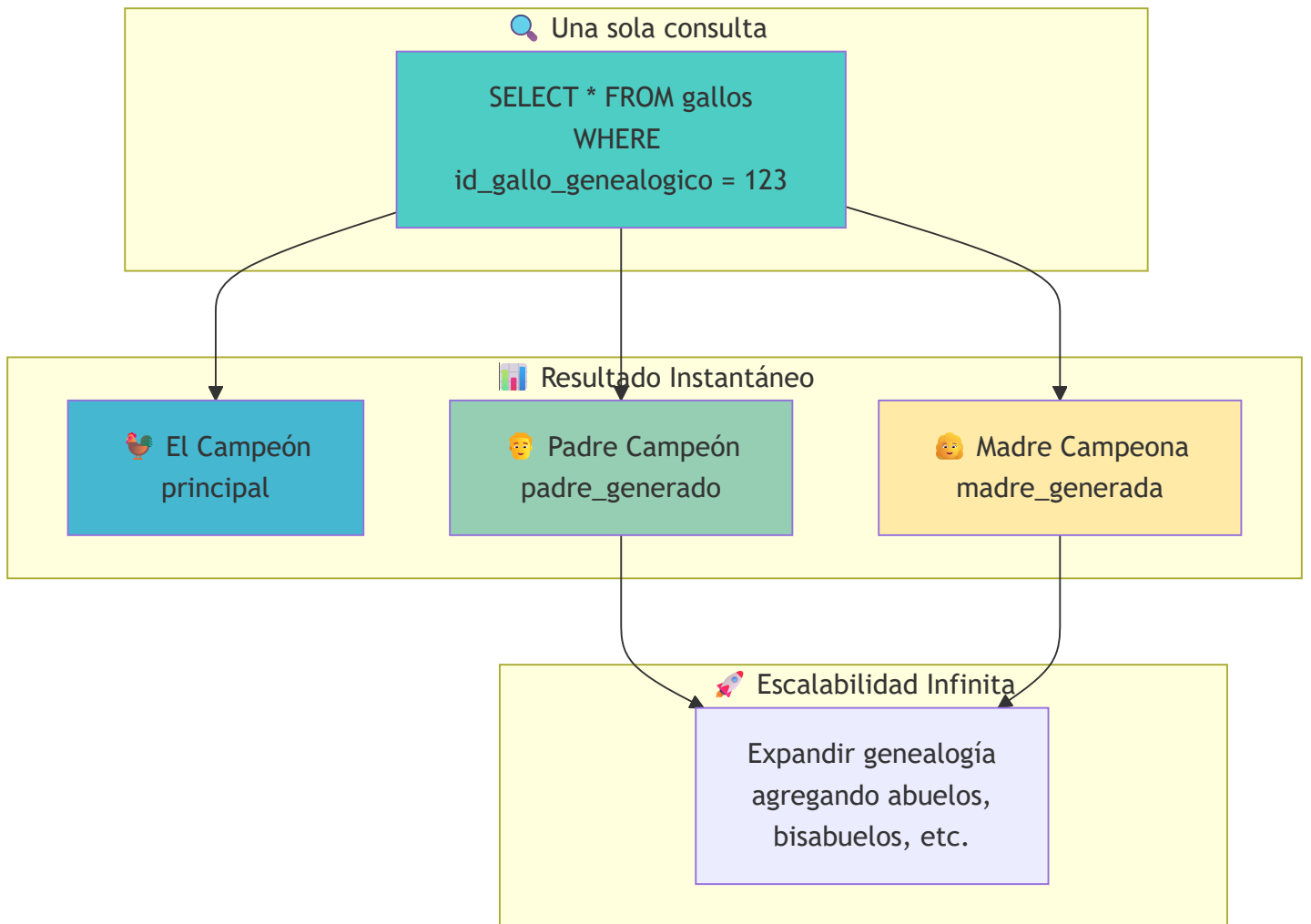
 Crear madre
tipo: 'madre_generada'

 Vincular relaciones
padre_id, madre_id

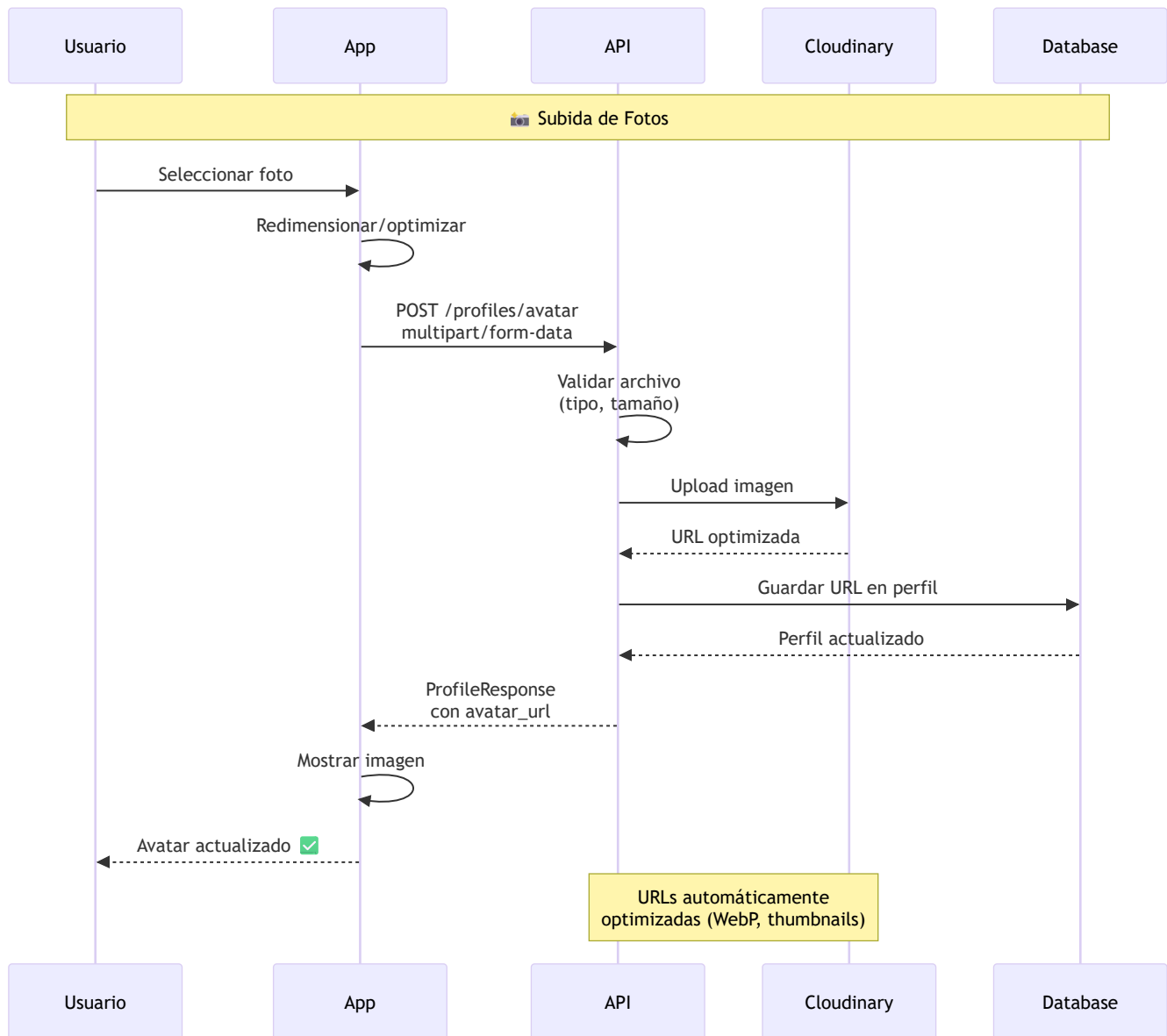
 RESULTADO:
1 request → 3 registros
Familia completa creada



Consulta Genealógica Mágica



Flujo de Gestión de Fotos



Endpoints Disponibles

Autenticación

Método	Endpoint	Descripción
POST	/auth/register	Registrar nuevo usuario
POST	/auth/login	Login con JWT

Método	Endpoint	Descripción
POST	/auth/refresh	Renovar access token
GET	/auth/me	Obtener perfil actual
POST	/auth/logout	Logout seguro
PUT	/auth/change-password	Cambiar contraseña

Perfiles

Método	Endpoint	Descripción
GET	/profiles/me	Mi perfil
PUT	/profiles/me	Actualizar perfil
POST	/profiles/avatar	 Subir avatar a Cloudinary
DELETE	/profiles/avatar	Eliminar avatar
GET	/profiles/me/complete	Perfil completo con usuario

Gallos

Método	Endpoint	Descripción
GET	/api/v1/gallos	Listar gallos con filtros
POST	/api/v1/gallos	Crear gallo básico
GET	/api/v1/gallos/{id}	Detalle completo del gallo
PUT	/api/v1/gallos/{id}	Actualizar gallo
DELETE	/api/v1/gallos/{id}	Eliminar gallo

Pedigrí (Técnica Épica)

Método	Endpoint	Descripción
POST	/api/v1/gallos/con-pedigri	 1 → 3 registros automáticos

Método	Endpoint	Descripción
GET	/api/v1/gallos/{id}/genealogia	Árbol genealógico completo

Fotos de Gallos

Método	Endpoint	Descripción
POST	/api/v1/gallos/{id}/foto	Subir foto del gallo
GET	/api/v1/gallos/{id}/fotos	Listar todas las fotos
DELETE	/api/v1/gallos/{id}/fotos/{numero}	Eliminar foto específica

Sistema

Método	Endpoint	Descripción
GET	/health	Health check general
GET	/test-db	Test conexión PostgreSQL
GET	/test-cloudinary	Test conexión Cloudinary
GET	/docs	 Swagger UI Interactivo



Ejemplos de Uso con cURL

1 Registrar Usuario

```
curl -X POST http://localhost:8000/auth/register \
-H "Content-Type: application/json" \
-d '{
  "email": "testcasto@galloapp.com",
  "password": "CastoGallo123@",
  "nombre_completo": "Usuario Test Casto",
  "telefono": "987654321",
  "nombre_galpon": "Galpon Casto Test",
  "ciudad": "Lima"
}'
```

2 Login y Obtener JWT

```
curl -X POST http://localhost:8000/auth/login \
-H "Content-Type: application/json" \
-d '{
  "email": "testcasto@galloapp.com",
  "password": "CastoGallo123@"
}'
```

3 Crear Gallo con Pedigrí (🔥 TÉCNICA ÉPICA)

```
curl -X POST http://localhost:8000/api/v1/gallos/con-pedigri \
-H "Authorization: Bearer YOUR_JWT_TOKEN" \
-H "Content-Type: application/x-www-form-urlencoded" \
-d "nombre=Campeón con Pedigrí&codigo_identificacion=PED001&peso=2.8&altura=48&color=Giro&crea"
```

4 Subir Avatar

```
curl -X POST http://localhost:8000/profiles/avatar \
-H "Authorization: Bearer YOUR_JWT_TOKEN" \
-F "file=@/ruta/a/imagen.jpg"
```

Configuración del Sistema

Variables de Entorno (.env)

```
# 🗄️ PostgreSQL Railway
DATABASE_URL=postgresql://postgres:KfktiHjbugWTzvalfwxiVZwsvVFatrk@gondola.proxy.rlwy.net:5416?sslmode=require

# 🔒 JWT
SECRET_KEY=galloapp-super-secret-key-development
ACCESS_TOKEN_EXPIRE_MINUTES=30

# 📁 Cloudinary
CLOUDINARY_CLOUD_NAME=dz4czc3en
CLOUDINARY_API_KEY=455285241939111
CLOUDINARY_API_SECRET=1uzQrkFD1Rbj8vPOClFBUEIwBn0

# 🌐 CORS
ALLOWED_HOSTS=["*"]

# 🔄 Environment
ENVIRONMENT=local
```

Ejecutar el Servidor

```
# Activar entorno virtual
venv\Scripts\activate

# Instalar dependencias
pip install -r requirements.txt

# Ejecutar servidor
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```



Esquema de Base de Datos

Tabla: usuarios

```
CREATE TABLE users (  
  id SERIAL PRIMARY KEY,  
  email VARCHAR(255) UNIQUE NOT NULL,  
  password_hash VARCHAR(255) NOT NULL,  
  is_active BOOLEAN DEFAULT true,  
  is_verified BOOLEAN DEFAULT true,  
  is_premium BOOLEAN DEFAULT false,  
  last_login TIMESTAMP,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Tabla: profiles

```
CREATE TABLE profiles (  
  id SERIAL PRIMARY KEY,  
  user_id INTEGER REFERENCES users(id),  
  nombre_completo VARCHAR(255) NOT NULL,  
  telefono VARCHAR(20),  
  nombre_galpon VARCHAR(255),  
  ciudad VARCHAR(100),  
  avatar_url TEXT, -- 📷 URL de Cloudinary  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Tabla: gallos (🔥 CON TÉCNICA RECURSIVA)

```
CREATE TABLE gallos (  
  id SERIAL PRIMARY KEY,  
  user_id INTEGER REFERENCES users(id),  
  raza_id INTEGER REFERENCES razas(id),  
  
  -- 🐔 Información básica  
  nombre VARCHAR(255) NOT NULL,  
  codigo_identificacion VARCHAR(50) NOT NULL,  
  peso NUMERIC(5,2),  
  altura INTEGER,  
  color VARCHAR(100),  
  estado VARCHAR(20) DEFAULT 'activo',  
  
  -- 🧬 TÉCNICA RECURSIVA GENEALÓGICA  
  id_gallo_genealogico INTEGER, -- 🔥 CAMPO MÁGICO  
  padre_id INTEGER REFERENCES gallos(id),  
  madre_id INTEGER REFERENCES gallos(id),  
  tipo_registro VARCHAR(20) DEFAULT 'principal',  
  
  -- 📁 Fotos Cloudinary  
  foto_principal_url TEXT,  
  url_foto_cloudinary TEXT,  
  fotos_adicionales JSONB,  
  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
-- 🚀 Índices para performance épica  
CREATE INDEX idx_gallos_genealogico ON gallos(id_gallo_genealogico);  
CREATE INDEX idx_gallos_padre ON gallos(padre_id);  
CREATE INDEX idx_gallos_madre ON gallos(madre_id);
```

🏆 Ventajas de la Técnica Recursiva

⚡ Performance

- **Una sola consulta** obtiene toda la familia

- **Índices optimizados** para búsquedas rápidas
- **Escalable** a millones de registros

Usabilidad

- **1 formulario** → **3 registros** automáticamente
- **Expansión fácil** de genealogía
- **Búsqueda intuitiva** por familias

Robustez

- **Validaciones completas** anti-ciclos
- **Transacciones seguras** con rollback
- **Error handling** robusto

URLs del Sistema

Ambiente	URL	Estado
Local	http://localhost:8000	✓ Funcionando
Railway	https://gallerappback-production.up.railway.app	✓ Funcionando
Swagger Local	http://localhost:8000/docs	 Documentación

Conclusión

CASTO_DE_GALLOSAPP API es un sistema completo y robusto que incluye:

- ✓ **Autenticación JWT** segura
- ✓ **Gestión de perfiles** con Cloudinary
- ✓ **Técnica pedigrí épica** (1 → 3 registros)
- ✓ **Genealogía recursiva** infinita
- ✓ **Gestión de fotos** optimizada
- ✓ **Base de datos** PostgreSQL Railway
- ✓ **API documentada** con Swagger

¡El sistema está listo para producción! 🚀 🐓

Desarrollado con ❤️ por el equipo de Casto de Gallos